CSCI 232 Assignment 04 – Linked List in C

Points: 10

Instructions

1. **Fork** the repository on GitHub: `https://github.com/jpach-cs/232-ass04-template`
2. **Clone** your forked repository to your local machine.
3. Open `src/code.c` and `src/tests.c`.
4. Uncomment the author block at the top of `src/code.c` and fill in your name:

```
// char *AUTHOR_NAME       = "Your Name";
// char *AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this
// assignment independently...";
```

5. Work through each function in **two parts**:

Part 1 — `src/code.c`

- Read the description and hint for each function.
- Implement the function body.
- Do **not** modify function signatures.
- Private functions are marked `static` — implement them first, they are called internally by public functions.

Part 2 — `src/tests.c`

- Read the description for each test function.
- Implement the test body using `TEST_ASSERT_TRUE_MESSAGE`.
- Stack tests: declare `Node` variables directly — no `malloc`.
- Heap tests: use `createNode` and `destroyList` for cleanup.
- Do **not** modify function names or signatures.

6. Build and run locally:

```
make
bin/main.exe
```

7. When all local tests pass — **commit** and **push** to your repository.
8. Go to the **Actions** tab on GitHub and verify the CI results.

Private functions — `src/code.c`

These are `static` — internal helpers, not visible outside `code.c`. Implement them first — public functions depend on them.

Function	Description
`_nullify`	Set pointer to NULL via double pointer
`_findFirst`	Return the first node (head)
`_findLast`	Walk to the last node and return it
`_findValue`	Walk and return first node matching value

Public functions — `src/code.c`

Function	Description
`initNode`	Initialize an existing node — set value, clear nextPtr
`createNode`	Allocate a new node on the heap via `malloc`
`destroyNode`	Free a heap node and set pointer to NULL
`destroyList`	Free all nodes in the list one by one
`addFirst`	Insert node at the front of the list
`addLast`	Insert node at the end of the list
`detachFirst`	Remove and return the first node — no free
`detachLast`	Remove and return the last node — no free
`detachValue`	Find, remove and return a node by value — no free
`deleteFirst`	Remove and free the first node
`deleteLast`	Remove and free the last node
`deleteValue`	Find, remove and free a node by value
`printList`	Print all values one per line
`listLength`	Return the number of nodes in the list

`detach` = remove from list, caller decides memory. `delete` = remove from list AND free — heap only. `delete` is always: `detach` + `destroyNode`.

Tests to implement — `src/tests.c`

Stack tests — use `Node` variables on the stack

Test function	What to verify
`test_initNode_sets_value`	value field contains the given value
`test_initNode_sets_next_null`	nextPtr is NULL after call
`test_initNode_null_guard`	NULL pointer does not crash
`test_detachFirst_returns_node`	returned pointer equals head node
`test_detachFirst_updates_head`	headPtr moves to second node
`test_detachFirst_empty_list`	returns NULL for empty list

Test function	What to verify
`test_detachValue_found`	correct node returned, chain reconnected
`test_detachValue_head`	head node correctly detached
`test_detachValue_not_found`	returns NULL when value missing

Heap tests — use `createNode` and `destroyList`

Test function	What to verify
`test_createNode_not_null`	returned pointer is not NULL
`test_createNode_value`	value field contains the given value
`test_createNode_next_null`	nextPtr is NULL after call
`test_destroyNode_sets_null`	pointer is NULL after destroyNode
`test_addFirst_empty_list`	headPtr points to new node
`test_addFirst_non_empty`	new node is at the front
`test_addFirst_null_headptr`	returns -1 when headPtrPtr is NULL
`test_addLast_empty_list`	headPtr points to new node
`test_addLast_non_empty`	new node is at the end
`test_addLast_null_guard`	returns -1 when headPtrPtr is NULL
`test_deleteFirst_removes_node`	returns 0, headPtr updated
`test_deleteFirst_empty_list`	returns -1 for empty list
`test_deleteValue_found`	returns 0, list length decreases
`test_deleteValue_not_found`	returns -1, list unchanged
`test_destroyList_empties_list`	headPtr is NULL after call
`test_listLength_empty`	returns 0 for NULL
`test_listLength_three`	returns 3 for three-node list
`test_printList_empty`	returns -1 for empty list

Rules

- Do **not** modify function signatures in `code.c` or `tests.c`
- Do **not** call private `_functions` from `tests.c`
- Stack tests: no `malloc`, no `free`
- Heap tests: always clean up with `destroyList` or `destroyNode`
- CI runs your `tests.c` together with instructor tests — both must pass

---

Grading

Component	Points
All functions correctly implemented	5
All test functions correctly written	5
Total	**10**

Author block — required

```
char *AUTHOR_NAME       = "Your Name";
char *AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this
assignment independently, except where explicitly noted and referenced.
Any collaboration or use of external resources has been properly cited.
I am fully aware of the consequences of academic dishonesty and agree
to abide by the university's academic integrity policy.";
```